

SISTEMA DE CHAT DISTRIBUÍDO

Nomes: Gustavo de Assis e Thiago Ribeiro

Link para o repositório do projeto:

OBS: Este relatório tem muitas imagens e um passo a passo, então excederá o número de páginas previstas.

1. INTRODUÇÃO

Este relatório tem como objetivo descrever o desenvolvimento de um Sistema de Chat Distribuído, projetado com microsserviços e que atende às seguintes demandas: comunicação em tempo real, acoplamento fraco, alta disponibilidade e escalabilidade horizontal.

Para a implementação do sistema de chat distribuído foi utilizada a linguagem Python, por ela oferecer bibliotecas e frameworks que facilitam a comunicação entre microsserviços, a criação e o gerenciamento de WebSockets, assim como a comunicação dos serviços com os bancos de dados.

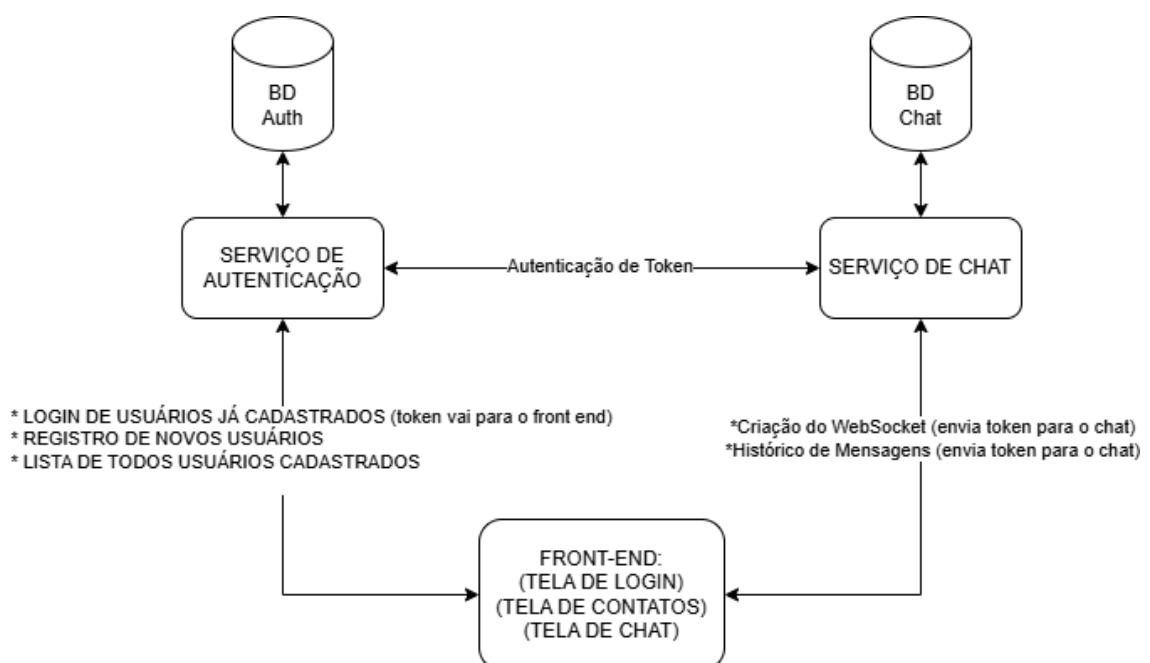
1.1. Descrição do sistema e dos serviços

Para a implementação desse sistema, a princípio foi criado um microsserviço de autenticação, um microsserviço de chat e o front-end que interliga os dois. Em termos gerais, o sistema consiste na autenticação do usuário que, quando realizada, encaminha para uma tela com todos os usuários do sistema. Essa tela permite selecionar um contato para conversar, o que é feito através do sistema de chat. Mostrar todos os usuários cadastrados é uma simplificação de um sistema real, onde cada usuário teria uma lista própria de contatos. Porém, como este é um sistema para fins de estudo, adotamos essa abordagem. Além disso, tanto o microsserviço de autenticação quanto o de chat possuem bancos de dados próprios.

O microsserviço de autenticação tem como objetivo gerenciar os usuários do sistema, podendo registrar novos usuários no banco de dados ou efetuar o login de usuários já existentes. Ao efetuar o login, ele gera um token que é enviado ao front-end, que usa esse token para acessar o serviço de chat. Além disso, o microsserviço de autenticação possui um serviço para verificar a autenticidade de um token recebido, cujo tempo de vida é previamente definido, para validação do token que o chat recebe do front, junto a isso, há uma função de gerar uma lista com todos os usuários existentes no banco.

O serviço de chat oferece dois serviços **principais**: um para baixar o histórico de uma conversa e outro para abrir o WebSocket. Ao selecionar um usuário para iniciar uma conversa no front-end, é enviada uma requisição ao serviço de chat para obter o histórico. Após isso, é iniciado um WebSocket entre o front-end e o chat. Ambos os serviços verificam a validade do token com o serviço de autenticação antes de prosseguir. Após a conexão WebSocket ser aberta, mensagens podem ser trocadas normalmente.

Uma visualização **simplificada** do fluxo do sistema é dada a seguir (ainda sem suporte para balanceamento de carga e escalabilidade horizontal):

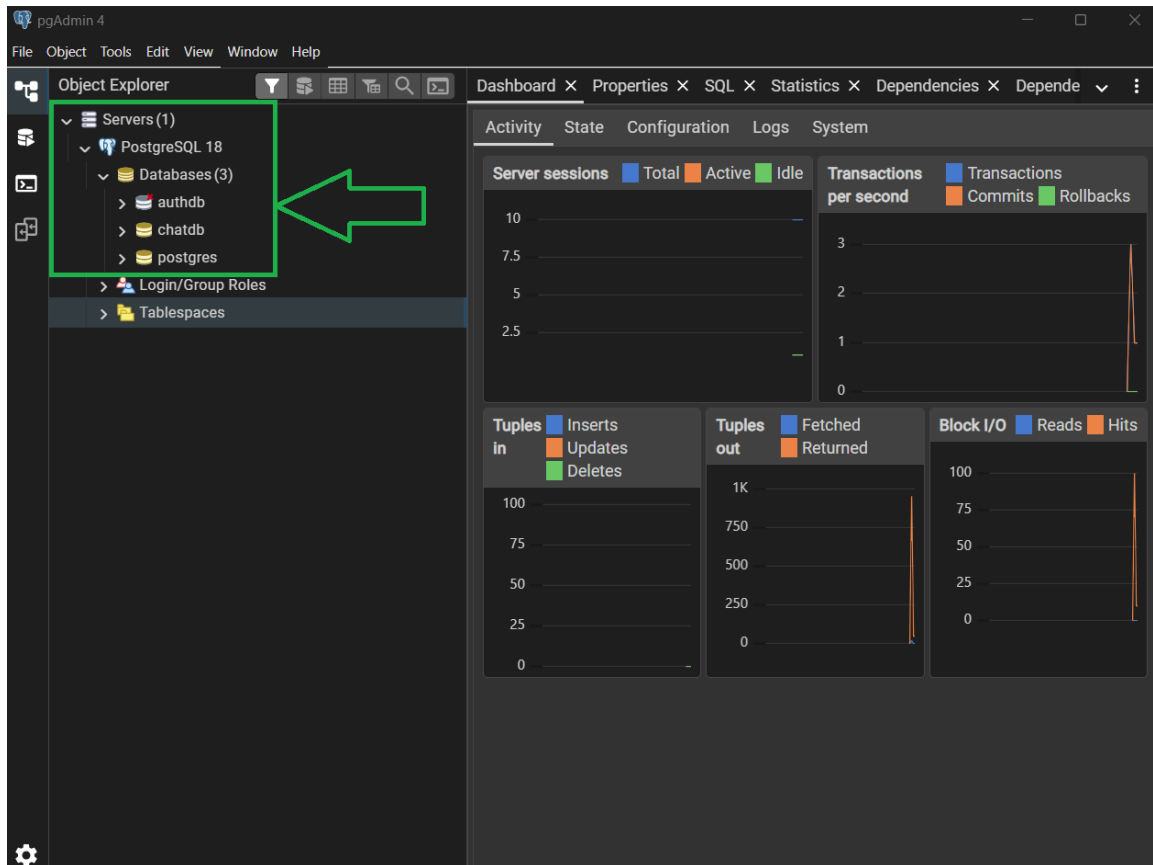


Há mais dois módulos que serão apresentados neste documento que irão permitir o balanceamento de carga e a escalabilidade horizontal do sistema.

2. BANCO DE DADOS:

O banco de dados utilizado foi o [PostgreSQL](https://www.postgresql.org/), um sistema de gerenciamento de banco relacional (SGBD) open-source, robusto e que já lida com concorrência. Para sua instalação (no Windows), basta acessar o site oficial e baixar o executável. Durante a instalação, instale todos os serviços indicados. Para criar os bancos:

1. Abra o programa pgAdmin 4.
2. Na aba lateral esquerda clique em Servers, PostgreSQL8, DataBases. (Você terá que criar uma senha durante o uso do programa, para acessar o banco de dados)



3. Clique com botão direito em Databases e vá em create → Database.
4. Dê um nome ao banco de dados e salve. No arquivo .env do projeto, altere a URL, usuário e senha conforme sua instalação. Exemplo:
 DATABASE_URL=postgresql://postgres:123456@localhost:5432/authdb
 SECRET_KEY=159753
 ALGORITHM=HS256
 ACCESS_TOKEN_EXPIRE_MINUTES=60

2.1. Modelos de dados

Nesse sistema de chat o modelo salvo no banco de autenticação é composto por:

- id
- nome
- senha (hash)

Enquanto o modelo do chat é composto por:

- id
- remetente
- destinatário
- conteúdo
- hora que foi enviado

3. FUNCIONAMENTO DO CHAT E ESTRUTURAS PARA ESCALABILIDADE HORIZONTAL

A partir da versão simplificada do sistema, apresentada no tópico 1.1, foi necessário a criação de mais dois módulos: um **canal global para comunicação entre os serviços de chat** e o **balanceador de carga**. Esses módulos são essenciais quando o sistema possui mais de um serviço de chat em execução simultaneamente.

O serviço de chat, em resumo, estabelece conexões WebSocket com o front-end de cada cliente. O sistema de chat pode possuir inúmeros WebSockets ativos com diferentes clientes, sendo responsabilidade deste serviço gerenciar essas conexões.

Quando um cliente A envia uma mensagem para um cliente B, o serviço de chat verifica se existe um WebSocket ativo com o cliente B. Se existir, a mensagem é enviada diretamente através dele. Se não existir, a mensagem fica salva no banco de dados e, quando o cliente B entrar no chat com A, todas as mensagens antes enviadas serão carregadas automaticamente.

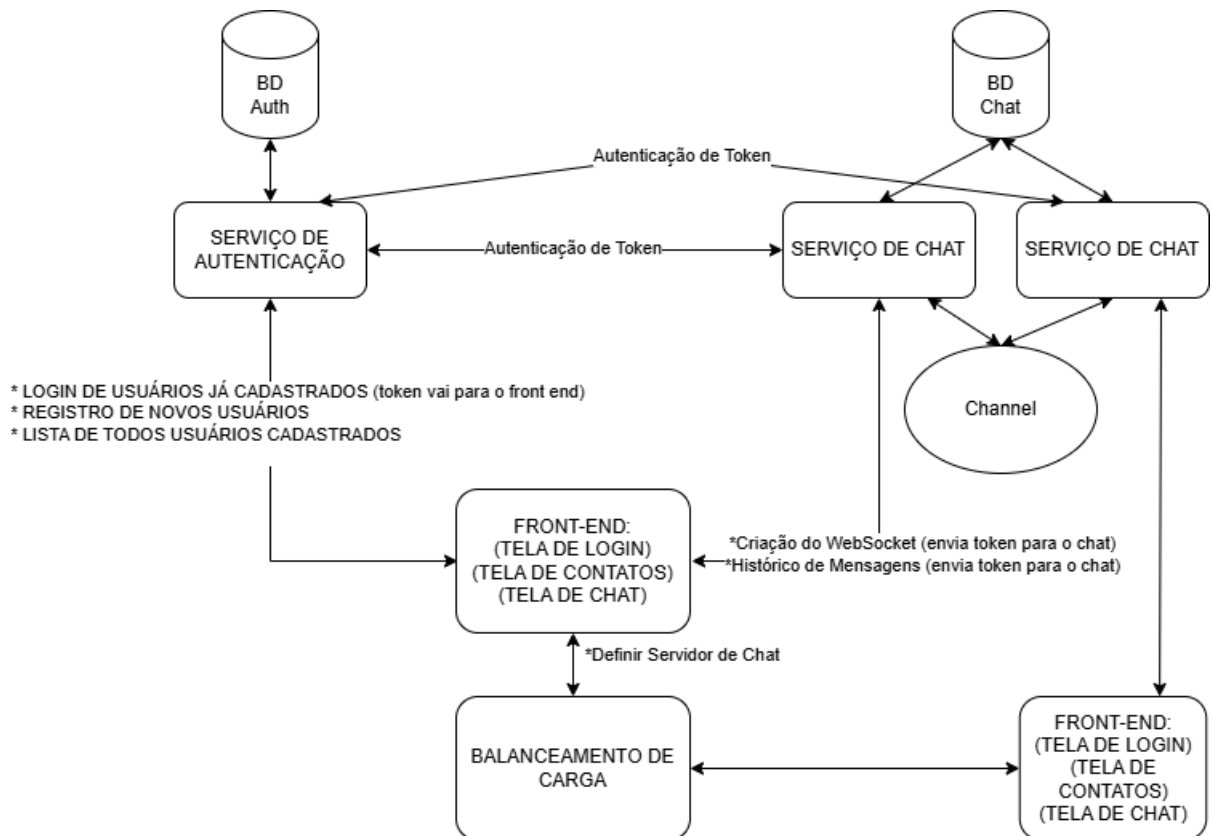
Quando temos múltiplos serviços de chat em atividade, é possível que cada cliente abra conexão com um servidor diferente. Por exemplo, o cliente A pode estar conectado ao chat 1 e o cliente B ao chat 2. Neste caso, a lógica apresentada até o momento não pode ser aplicada, por que a gestão dos websockets estão sendo feitas em serviços diferentes. Em suma, o serviço de chat 1 não sabe quais websockets estão ativos no serviço de chat 2, e vice versa.

Para resolver esse problema, foi criado um **canal global para comunicação entre os serviços de chat**. Esse canal tem websockets abertos com todos os serviços de chat disponíveis, servindo para repassar requisições entre eles. Agora, basicamente, quando um cliente A em um serviço de chat 1 envia mensagem para um cliente B em um serviço de chat 2, após o serviço de chat 1 perceber que não tem um websocket ativo com o cliente B (pois ele está em outro servidor) vai mandar uma requisição no canal global, onde ele vai repassar para todos os demais servidores a pergunta “alguém tem websocket ativo com o cliente B? Se sim, envie esta mensagem <corpo da mensagem> para ele”, o chat 2 ao receber a pergunta e ver que possui um websocket ativo com o cliente B, vai enviar a mensagem para ele.

Além disso, outro problema surge com múltiplos serviços de chat: com qual servidor cada cliente deve se conectar? Para isso, criamos o balanceador de carga, um serviço que o front-end acessa antes de abrir a conexão com o chat. O balanceador possui as URLs de cada

serviço de chat e faz uma seleção round-robin entre eles, distribuindo os clientes igualmente. Se existirem N serviços ativos, cada novo cliente é direcionado para o próximo servidor da lista.

O diagrama do processo com suporte a escalabilidade horizontal é dado por:



A escalabilidade neste projeto foi implementada somente para o serviço de chat, o qual é mais complexo e necessita de uma lógica a mais para sua implementação. A lógica para implementar mais de um serviço de autenticação é simples, basta adicionar servidores de autenticação e no balanceador de carga destinar o acesso para cada um, junto a isso, passar a URL deste serviço para o chat para ele fazer a verificação do token diretamente com esse serviço. Enfim, devido a sua simplicidade, foi acordado que não havia necessidade de implementação, mas sim uma menção simples, que aqui foi realizada.

4. FRONT-END

Com relação ao *design* das páginas: as telas de login, contatos e chat foram feitas com HTML e CSS, sendo constituídas por estruturas simples em tons de azul e branco.

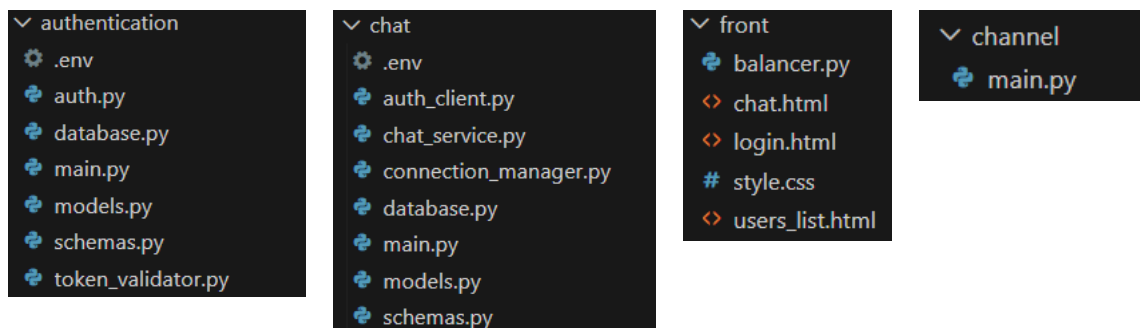
The image displays three distinct user interface screens for a web application, all featuring a clean design with a light gray background and blue accents.

- Login / Registro:** This screen contains two input fields labeled 'Usuário' and 'Senha'. Below these fields are two blue buttons: 'Entrar' and 'Registrar'.
- Lista de Usuários:** This screen displays a list of four users, each represented by a blue button with the user's name: 'Gustavo', 'Peixe', 'Astronauta', and 'StarMan'.
- Chat:** This screen is titled 'Chat' and indicates the current conversation partner as 'Conversando com: Astronauta'. It features a large white rectangular area for the chat messages, showing a message from 'Gustavo: Olá'. At the bottom, there is a text input field labeled 'Mensagem' and a blue 'Enviar' button.

A lógica do front-end é constituída por scripts em JavaScript inseridos nos arquivos HTML. Os scripts chamam funções dos microserviços, incluindo o balanceador de carga. O front-end também mantém variáveis no armazenamento local, como o token de autenticação.

5. ESTRUTURA DO CÓDIGO

A estrutura do sistema de chat é composta pelos módulos:



No diretório de *authentication* temos um arquivo `.env` que guarda a URL do banco de dados, sua senha, o algoritmo usado e uma variável que define o tempo de expiração dos tokens. Além disso, temos a classe *auth* que realiza a autenticação e registro no banco de dados, assim como a criação do token de acesso para o chat, *database* declara o banco de dados, *main* contém as rotas de acesso do serviço, *models* define o modelo dos dados armazenados no banco, *schemas* define o formato dos dados trocados entre os serviços, e por fim, o *token_validator*, que valida os tokens.

No *chat* temos uma estrutura semelhante. A classe *auth_client* faz a comunicação com o serviço de autenticação para validar o token recebido pelo front, *chat_service* faz a lógica de comunicação com o banco de dados, *connection_manager* lida com as conexões WebSocket (mantendo um dicionário de quem está conectado com quem), *database*, *models* e *schemas* funcionam como no serviço de autenticação. A *main* define as rotas (histórico e criação de WebSocket). A *main* deste microserviço também é responsável pela comunicação com o novo módulo, o *channel*.

Em *channel* temos uma única classe que é responsável por aceitar as conexões websockets do serviços de chat e reencaminhar mensagens.

O *front* é constituído de documento HTML, um CSS o *balanceador* que foi colocado no mesmo diretório por simplicidade.

Para a criação das rotas utilizou-se o framework **FastAPI**. O FastAPI é um framework Python que facilita tanto a construção de APIs REST quanto o gerenciamento de conexões WebSocket. Além disso, ele possui suporte nativo a funções assíncronas (`async/await`). Foram utilizadas principalmente as bibliotecas Jose para lidar com tokens, Pydantic para formatação dos dados e SQLAlchemy para conexão com o banco de dados.

6. USABILIDADE DA PÁGINA

Com relação a usabilidade da página: a página gera um aviso de erro se você tentar logar com um usuário inexistente ou tentar registrar um usuário já existente. Se o cliente tentar acessar a URL da página de `users_list` ou chat diretamente, ele será automaticamente redirecionado para a página de login (verificação simples se há token armazenado no navegador). Se o token expirar ou o sistema cair durante o uso, o cliente também será automaticamente redirecionado para a página de login.

7. ATENDENDO OS OBJETIVOS DO TRABALHO

Os objetivos deste trabalho são destacar alguns dos principais aspectos de um sistema de chat distribuído: alta disponibilidade, comunicação em tempo real e escalabilidade horizontal.

7.1. Alta Disponibilidade

A alta disponibilidade tem dois aspectos, primeiro a independência dos serviços: o serviço de chat, se já tiver sido autenticado, funciona mesmo se não existir nenhum serviço de autenticação em funcionamento, da mesma forma, o serviço de autenticação funciona sem a existência de nenhum serviço de chat ativo. O outro aspecto se baseia no balanceamento de carga, se existir mais de um serviço de chat ou mais de um serviço de autenticação, se um desses falhar a conexão, o cliente volta para o login e pode se conectar novamente em outro serviço de chat.

7.2. Comunicação em Tempo Real

A comunicação em tempo real se estabelece com o uso do websocket como descrito anteriormente. Com ele, não é necessário requisitar informações ao servidor mais, essa comunicação se torna mais fluída, com o servidor enviando as mensagens assim que as recebe.

7.3. Escalabilidade Horizontal

A escalabilidade horizontal consiste em podermos adicionar servidores para lidar com o número crescente de usuários. Essa escalabilidade se deve principalmente à independência dos microsserviços, podendo esses ser adicionados sem mudanças no funcionamento do sistema, sendo necessário apenas destinar os acessos para esses novos serviços através do balanceamento de carga.

8. MONITORAMENTO DE SERVIÇOS ATIVOS

O balanceador de carga, como dito anteriormente, é responsável em dizer com qual servidor de chat o front-end deve se conectar. Mas em um sistema distribuído, componentes podem apresentar falhas, e isso se aplica aos microsserviços.

Em um sistema com N serviços de chat, é plausível considerar que alguns estejam em falha em algum momento. O balanceador de carga do nosso projeto, tem como responsabilidade saber quais serviços estão em falha e quais estão ativos, para não direcionar um cliente para um serviço que não esteja ativo.

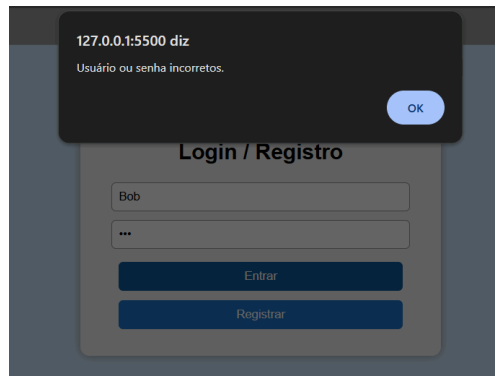
Para determinar os servidores ativos o balanceador envia um ping, de forma periódica, para todos os serviços de chat, se não houver resposta, ele determina que o módulo não está funcionando, se houver resposta, ele considera aquele serviço como ativo. Além do ping, se a conexão web-socket de algum cliente com o servidor for encerrada por algum motivo durante o uso do sistema, ele também determina que aquele módulo não está funcionando. Se um módulo que falhou voltar a responder o ping, ele volta para os servidores ativos podendo receber mais requisições.

9. TESTES

9.1. Testes Unitários

Com relação aos testes, começando com os **testes unitários**, que consistem em verificar se o sistema é capaz de cobrir a lógica de negócios central dos microsserviços, como a validação de credenciais de login e a persistência correta de mensagens no banco de dados.

A partir da tela inicial, se você tentar entrar com dados de um usuário ainda não cadastrado no banco, será gerado um erro.



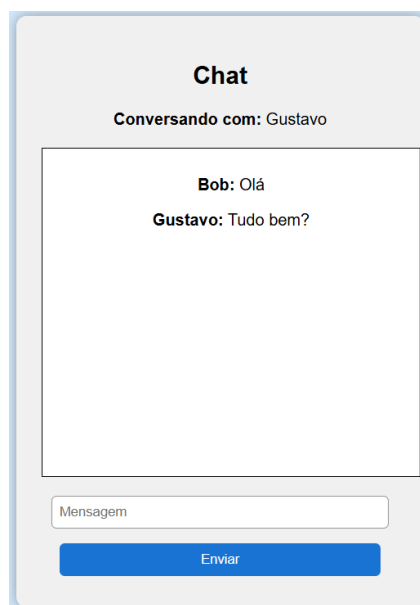
Da mesma forma, se você tentar registrar um usuário já cadastrado. Quando você cadastra um usuário novo, ele será salvo no banco de dados e você agora pode efetuar login com ele.

Entrando com um usuário válido, o sistema de autenticação irá aprovar sua entrada e você terá acesso a lista de contatos.

Este comportamento pode ser visto no terminal aberto do programa. Aqui está um log onde é feito uma tentativa errada de login e depois uma certa:

```
INFO: 127.0.0.1:58727 - "OPTIONS /login HTTP/1.1" 200 OK
INFO: 127.0.0.1:58727 - "POST /login HTTP/1.1" 400 Bad Request
INFO: 127.0.0.1:54488 - "OPTIONS /register HTTP/1.1" 200 OK
INFO: 127.0.0.1:54488 - "POST /register HTTP/1.1" 200 OK
INFO: 127.0.0.1:54488 - "POST /login HTTP/1.1" 200 OK
```

Ao selecionar um contato para conversar, você passa para o serviço de chat, onde ele baixa com sucesso as conversas que você já teve com aquele contato.



9.2. Testes de Integração

Continuando com os **testes de integração**, que consistem em verificar a comunicação correta e o fluxo de dados entre os componentes. Esses testes incluem:

- Comunicação entre o Serviço de Autenticação e o Serviço de Mensagens (ex: Autenticar e, em seguida, enviar uma mensagem).
- Comunicação do Front-end com os serviços de backend.

Como discutido anteriormente, o front obtém um token vindo do serviço de autenticação e envia para o serviço de chat, que verifica esse token com o serviço de autenticação para ver se está correto. Podemos mostrar essa requisição ocorrendo no terminal do serviço de autenticação:

```
Verificação do token pelo Chat
INFO:      127.0.0.1:55211 - "GET /verify-token HTTP/1.1" 200 OK
Verificação do token pelo Chat
INFO:      127.0.0.1:49721 - "GET /verify-token HTTP/1.1" 200 OK
```

Após isso, o serviço de chat funciona normalmente. Se houver a tentativa de passar um token inválido, a conexão web-socket não é aberta, ou seja, não há troca de mensagens, assim como o histórico não é baixado. Abaixo está o log do site quando é passado um token inválido do chat para o autenticador.

```
✖ Failed to load resource: the server responded with a status of localhost:8001/history/Peixe:1 401 (Unauthorized)
✖ Uncaught (in promise) TypeError: hist.forEach is not a function chat.html:67
    at loadHistory (chat.html:67:18)
    at async init (chat.html:50:13)
```

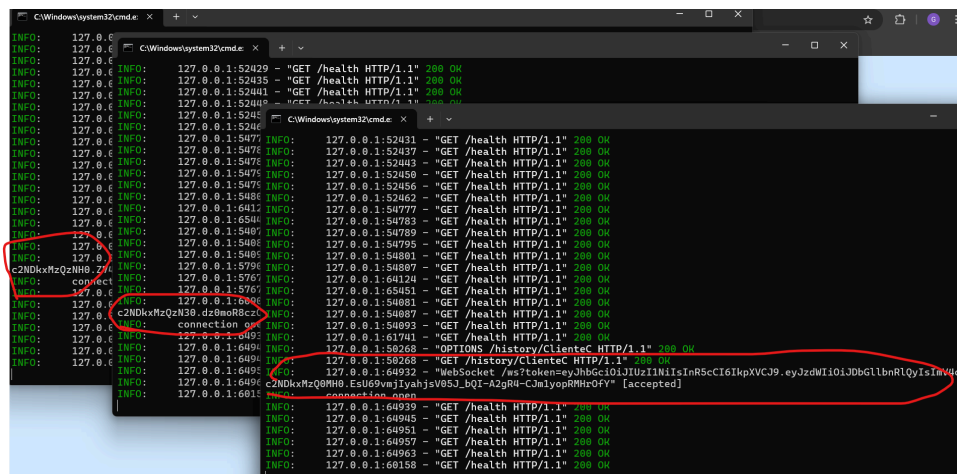
As demais comunicações entre front e back-end ficam evidenciadas nos demais testes.

9.3. Testes de Concorrência/Carga

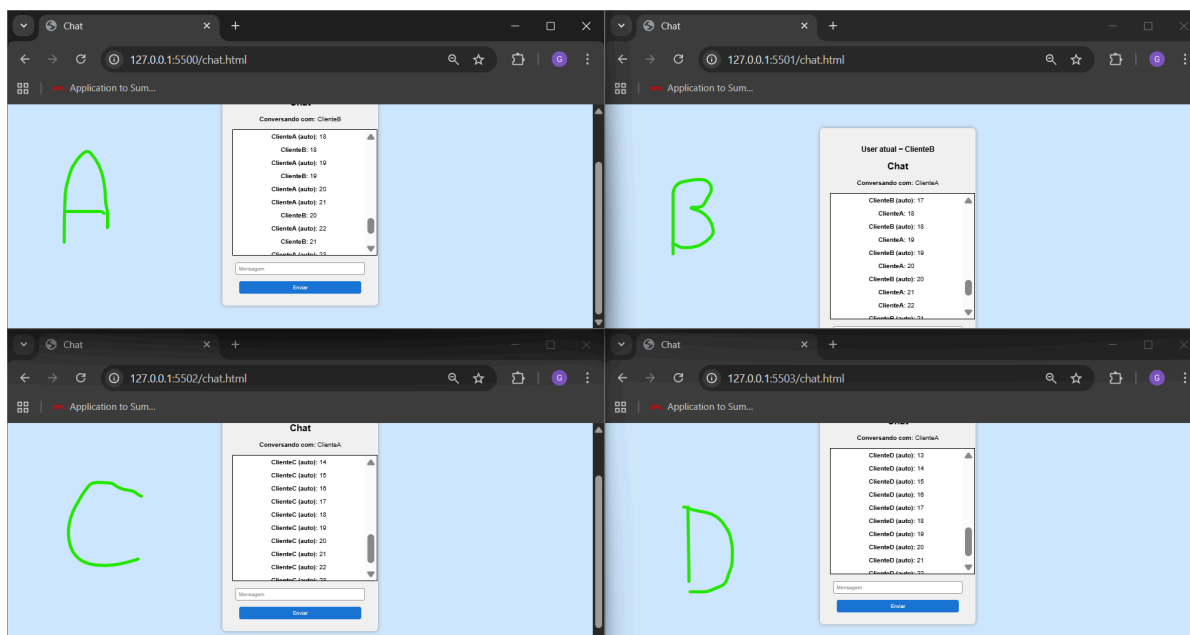
Por fim, os testes de **Concorrência/Carga**, que tem como objetivo provar a Escalabilidade Horizontal e o funcionamento eficaz do balanceamento de carga, podendo ser realizados através da simulação de múltiplos usuários (mínimo de 10 usuários simultâneos) fazendo login e trocando mensagens.

Para realizar esse teste, devemos criar vários clientes (um servidor front-end para cada). E criar vários serviços de chat (para cada chat criado se deve atualizar os endereços no arquivo python do balanceador).

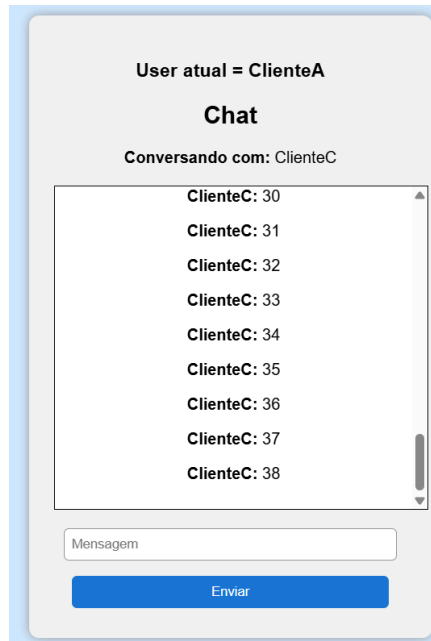
O primeiro ponto a se destacar é: existindo vários servidores, o balanceador encaminha cada conexão para cada servidor usando uma configuração round robin com os servidores ativos. Podemos ver a distribuição de conexões nos logs dos 3 servidores ativos para o teste depois de 3 dispositivos iniciarem um chat um após o outro:



Para realizar a troca de mensagens entre eles modificamos nosso script da página do chat para enviar mensagens automaticamente a cada 5 segundos um para os outros. Vamos pegar 4 usuários: A, B, C e D (enquanto isso, conversas entre os demais aconteciam de fundo) no que eu considero o pior caso, A no chat com B e B no chat com A, e por sua vez, C e D conversando com A. Este caso mostra a funcionalidade do sistema por completo, pois ele mostra que o histórico funciona, assim como o isolamento das conversas.



Tanto C como D enviam mensagens para A enquanto ele está no chat com B, mas essas mensagens não aparecem na tela, já que o chat é 1x1 e ele está no chat com B. Agora se acessarmos o chat de A com C, a mensagem enviada enquanto A estava em outro chat aparece normalmente:



Nesse processo não houve delay nem problemas de mensagens perdidas, isso indica que o funcionamento está funcionando corretamente quando há alta demanda.

10. CONCLUSÕES

O sistema foi implementado com sucesso, passando em todos os testes e conseguindo atingir todos os objetivos do trabalho.